

Group13
Players Profile and Gaming Platform

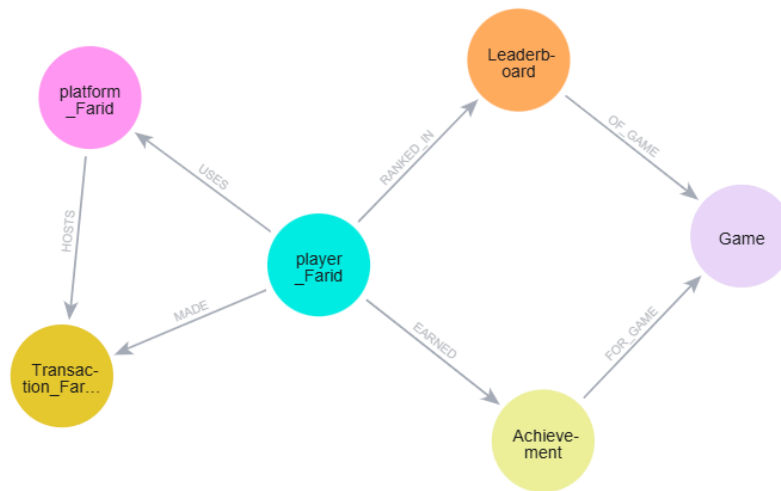
Farid Ahmad Tokhi
Alif Hossain

Contributions

Farid Ahmad Tokhi – Platforms, Transactions and Players

Alif Hossain – Achievements, Leaderboards, Game

Schema



SIMPLE QUERIES

Simple Query 1 (Farid Ahmad Tokhi):

```
MATCH (t:Transaction_Farid {transactionId: "T005"})
RETURN t;
```

Description:

This query returns the details of a single transaction, identified by its unique transactionId.

It show how to retrieve one specific record using filtering.

Output:

The screenshot displays the Neo4j Aura web interface. At the top, the user is logged in as 'fatokhi@myseneca.ca'. The query editor shows the following Cypher query:

```
1 MATCH (t:Transaction_Farid {transactionId: "T005"})
2 RETURN t;
```

The query is executed, and the results are displayed in the 'Graph' view. A single node is shown, representing the transaction with a cost of 12.99. The 'Node details' panel on the right provides a table of the transaction's properties:

Key	Value
<id>	4:46f7b3a7-7e83-42eb-ad9e-2e183ebb7bf9:44
cost	12.99
currency	"CAD"
dateBought	2024-06-30
gameId	5002
paymentMethod	"Debit Card"
playerId	1005
productBought	"DLC Map Pack"
status	"Completed"
transactionId	"T005"

Simple Query 2 (Farid Ahmad Tokhi):

```
MATCH (pl:player_Farid)-[:USES]->(pf:platform_Farid)
WHERE pf.platformName = "PlayStation Network"
RETURN pl.playerId, pl.username, pf.platformName;
```

Description:

This query finds all players who are connected to the PlayStation Network platform through the USES relationship.

Output:

Aura (fatokhi@myseneca.ca) ▼

neo4j\$

```
1 MATCH (pl:player_Farid)-[:USES]->(pf:platform_Farid)
2 WHERE pf.platformName = "PlayStation Network"
3 RETURN pl.playerId, pl.username, pf.platformName;
```

Table RAW

	pl.playerId	pl.username	pf.platformName
1	1007	"98aversegoblin"	"PlayStation Network"
2	1016	"94littletrombone"	"PlayStation Network"

Started streaming 2 records after 42 ms and completed after 43 ms.

Simple Query 3 (Farid Ahmad Tokhi):

```
MATCH (t:Transaction_Farid)
WHERE t.paymentMethod = "Credit Card"
RETURN t.transactionId, t.productBought, t.cost, t.paymentMethod
```

Description:

This query filters all transactions to return only those where the payment method is Credit Card.

Output:

Aura (fatokhi@myseneca.ca) neo4j\$

```
1 MATCH (t:Transaction_Farid)
2 WHERE t.paymentMethod = "Credit Card"
3 RETURN t.transactionId, t.productBought, t.cost, t.paymentMethod
```

Table RAW

	t.transactionId	t.productBought	t.cost	t.paymentMethod
1	"T001"	"Season Pass"	24.99	"Credit Card"
2	"T004"	"Premium Pass"	29.99	"Credit Card"
3	"T006"	"Battle Coins 50 00"	19.99	"Credit Card"
4	"T010"	"Weapon Bundle"	17.99	"Credit Card"
5	"T012"	"Premium Members hip"	49.99	"Credit Card"
6	"T014"	"Gold Pack 10,00 0"	25.99	"Credit Card"
7	"T016"	"Special Event T icket"	5.49	"Credit Card"
8	"T018"	"Character Pack"	22.99	"Credit Card"

ADVANCED QUERIES

Advanced Query 1 (Farid Ahmad Tokhi):

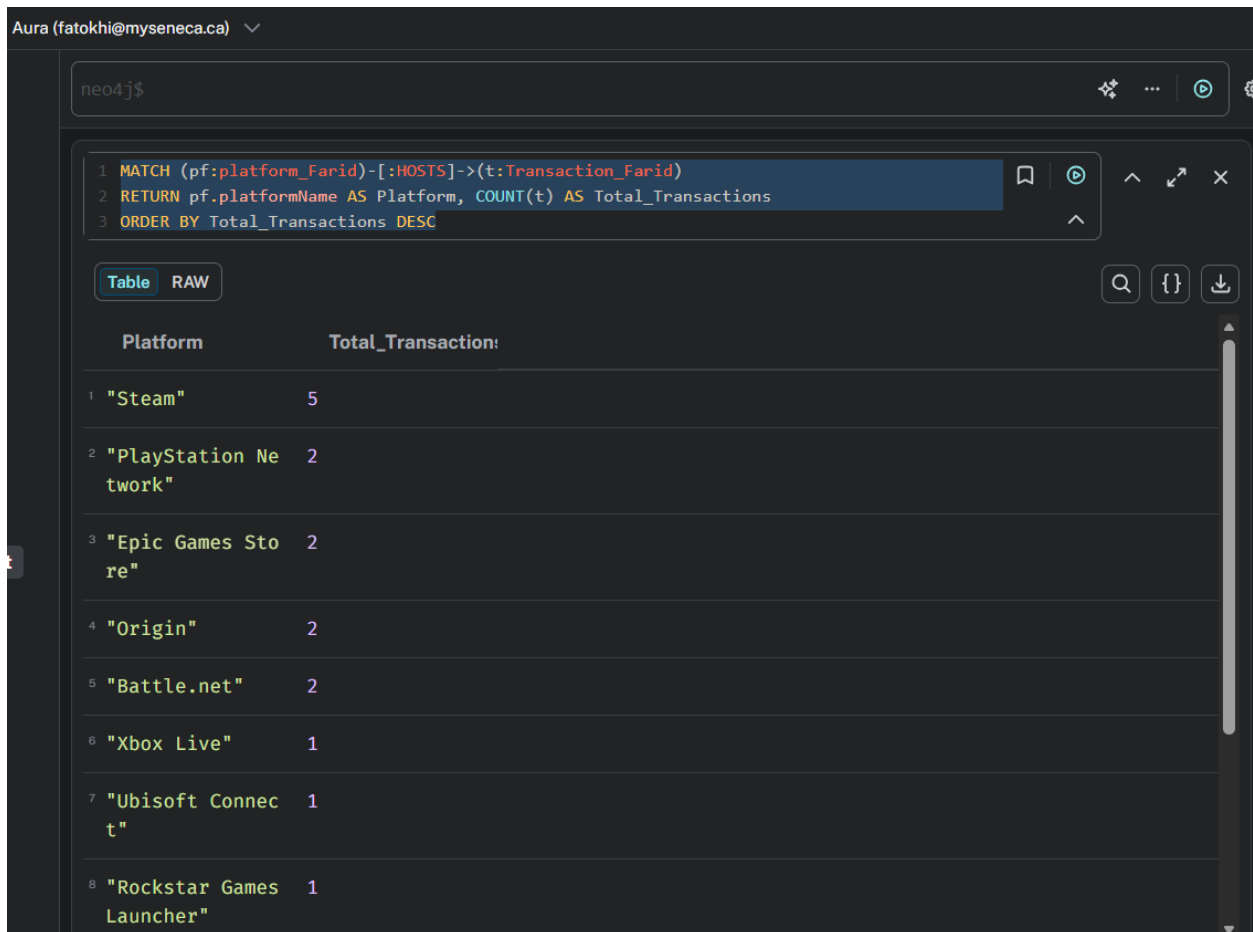
```
MATCH (pf:platform_Farid)-[:HOSTS]->(t:Transaction_Farid)
RETURN pf.platformName AS Platform, COUNT(t) AS Total_Transactions
ORDER BY Total_Transactions DESC
```

Description:

This query aggregates all transactions hosted by each platform.

It uses pattern matching and the COUNT () function to summarize how many purchases were made on every platform.

Output:



Aura (fatokhi@myseneca.ca) neo4j\$

```
1 MATCH (pf:platform_Farid)-[:HOSTS]->(t:Transaction_Farid)
2 RETURN pf.platformName AS Platform, COUNT(t) AS Total_Transactions
3 ORDER BY Total_Transactions DESC
```

	Platform	Total_Transactions
1	"Steam"	5
2	"PlayStation Network"	2
3	"Epic Games Store"	2
4	"Origin"	2
5	"Battle.net"	2
6	"Xbox Live"	1
7	"Ubisoft Connect"	1
8	"Rockstar Games Launcher"	1

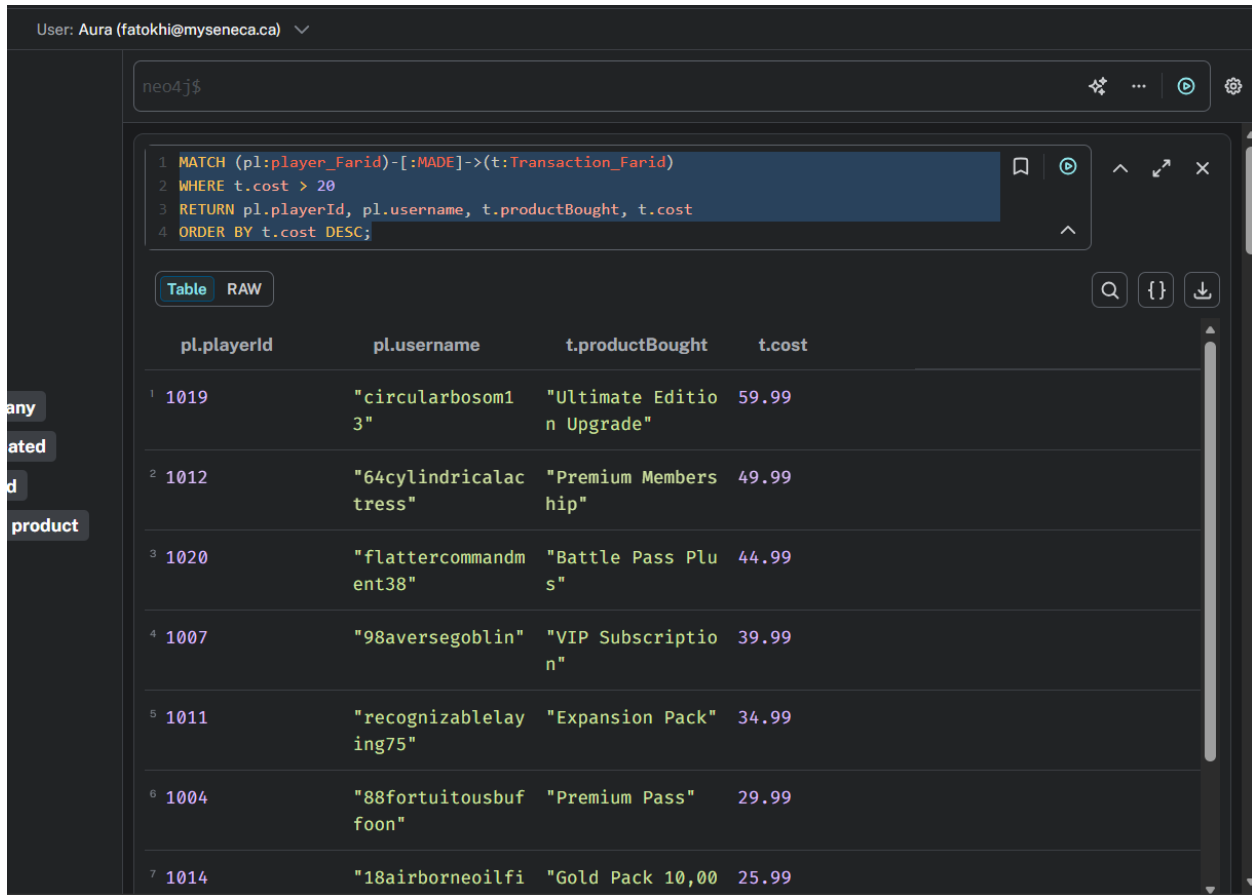
Advanced Query 2 (Farid Ahmad Tokhi):

```
MATCH (pl:player_Farid)-[:MADE]->(t:Transaction_Farid)
```

```
WHERE t.cost > 20
RETURN pl.playerId, pl.username, t.productBought, t.cost
ORDER BY t.cost DESC;
```

Description:

This query filters the transactions to find players who made purchases costing more than \$20.

Output:

The screenshot shows a Neo4j query editor interface. At the top, the user is identified as 'Aura (fatokhi@myseneca.ca)'. The query editor contains the following Cypher query:

```
1 MATCH (pl:player_Farid)-[:MADE]->(t:Transaction_Farid)
2 WHERE t.cost > 20
3 RETURN pl.playerId, pl.username, t.productBought, t.cost
4 ORDER BY t.cost DESC;
```

Below the query editor, the results are displayed in a table view. The table has four columns: 'pl.playerId', 'pl.username', 't.productBought', and 't.cost'. There are seven rows of data, ordered by 't.cost' in descending order.

	pl.playerId	pl.username	t.productBought	t.cost
1	1019	"circularbosom13"	"Ultimate Edition Upgrade"	59.99
2	1012	"64cylindricalactress"	"Premium Membership"	49.99
3	1020	"flattercommandment38"	"Battle Pass Plus"	44.99
4	1007	"98aversegoblin"	"VIP Subscription"	39.99
5	1011	"recognizablelaying75"	"Expansion Pack"	34.99
6	1004	"88fortuitousbuffoon"	"Premium Pass"	29.99
7	1014	"18airborneoilfi"	"Gold Pack 10,00"	25.99

Advanced Query 3 (Farid Ahmad Tokhi):

```
MATCH (pf:platform_Farid)<-[:USES]-(pl:player_Farid)-[:MADE]->(t:Transaction_Farid)
RETURN pf.platformName, pl.username, t.gameId, t.productBought, t.dateBought
ORDER BY pf.platformName, t.dateBought;
```

Description:

this traversal query combines data from three node types: platform, player, and transaction.

It shows which players on each platform made purchases, which game they relate to, and the time of purchase.

Output:

ser: Aura (fatokhi@myseneca.ca) ▼

neo4j\$

```
1 MATCH (pf:platform_Farid)<-[:USES]-(pl:player_Farid)-[:MADE]->(t:Transaction_Farid)
2 RETURN pf.platformName, pl.username, t.gameId, t.productBought, t.dateBought
3 ORDER BY pf.platformName, t.dateBought;
```

Table RAW

	pf.platformName	pl.username	t.gameId	t.productBought	t.dateBought
1	"Amazon Luna"	"38settledincarnation"	5018	"Battle Coins 5000"	2024-07-15
2	"Battle.net"	"88fortuitousbuffoon"	5005	"Premium Pass"	2024-05-13
3	"Battle.net"	"deepeningten63"	5012	"Weapon Bundle"	2024-08-18
4	"Epic Games Store"	"playingturn91"	5017	"Character Unlock"	2024-06-10
5	"Epic Games Store"	"fluttercommandment38"	5014	"Battle Pass Plus"	2024-08-28
6	"GOG Galaxy"	"agingsubstance47"	5019	"Weapon Upgrade"	2024-08-01
7	"NVIDIA GeForce NOW"	"geometricminiskirt61"	5009	"Skin Bundle"	2024-07-09

SIMPLE QUERIES (Alif)

Simple Query 1:

MATCH (g:Game)


```
RETURN g.game_ID, g.name, g.developer, g.publisher, g.date_published
LIMIT 10;
```

Description:

This query returns the details of a single game, identified by its unique game_ID.

It demonstrates how to retrieve one specific record from the Game node label by applying a simple equality filter.

Output:

The screenshot shows the Neo4j Aura interface. The left sidebar contains navigation options like 'Get started', 'Developer hub', 'Data services', 'Instances', 'Import', 'Graph Analytics', 'Data APIs', 'Agents', 'Tools', 'Query', 'Explore', 'Dashboards', 'Operations', 'Project', and 'Learning'. The main area displays 'Database information' with 'Nodes (80)' and 'Relationships (80)'. A query is entered in the console: `neo4j$ MATCH (g:Game {game_ID: 5001}) RETURN g;`. The results are shown in a table view, displaying a single record for the Game node with the following properties:

Key	Value
<id>	4:950c7043-7901-4864-8e96-82387434d738:0
date_published	2023-03-08
developer	"Infinity Ward"
game_ID	5001
name	"Call of Duty: Modern Warfare"
publisher	"Activision"

Below the table, a graph visualization shows a single node labeled 'Call of Duty: M...'. The console output at the bottom shows the query result in a JSON-like format: `neo4j$ UNWIND [ { leaderboard_id: 8001, game_ID: 5001, player_ID: 1001, w... } ]`. The status bar at the bottom indicates 'Created 40 relationships, set 20 properties' and 'Completed after 128 ms'.

Simple Query 2:

```
MATCH (g:Game {developer: "Nintendo"})
```

```
RETURN g;
```

Description:

This query retrieves all games created by a specific developer, in this case Nintendo. It demonstrates how to filter nodes based on a non-unique attribute to return multiple matching records.

Output:

The screenshot shows the Neo4j Aura console interface. On the left is a sidebar with navigation options like 'Get started', 'Developer hub', 'Data services', and 'Tools'. The main area is divided into three panels. The left panel, 'Database information', shows 80 nodes (Achievement, Game, Leaderboard, Player) and 80 relationships (EARNED, FOR_GAME, OF_GAME, RANKED_IN). The middle panel displays the query: `neo4j$ MATCH (g:Game {developer: "Nintendo"}) RETURN g;`. The right panel shows the results overview with 2 nodes and 2 games. The graph view shows two nodes labeled 'Super Mario...' connected by a relationship. The bottom panel shows the next query: `neo4j$ MATCH (g:Game {game_ID: 5001}) RETURN g;`.

Simple Query 3:

```
MATCH (p:Player {player_ID: 1001})-[:EARNED]->(a:Achievement)
```

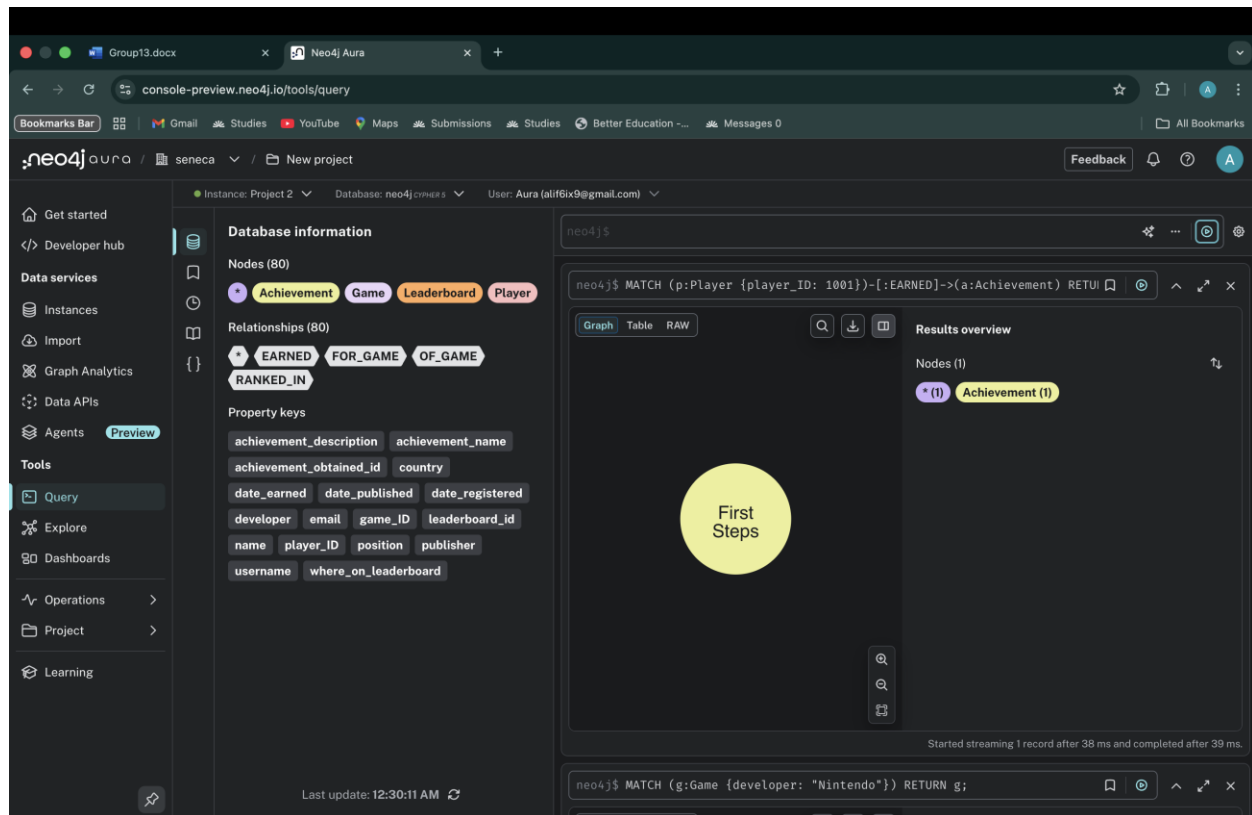
```
RETURN a;
```

Description:

This query returns all achievements earned by a specific player, identified by their unique `player_ID`.

It demonstrates how to traverse a relationship (EARNED) to retrieve connected records from another node label.

Output:



ADVANCED QUERIES (Alif)

Advanced query 1:

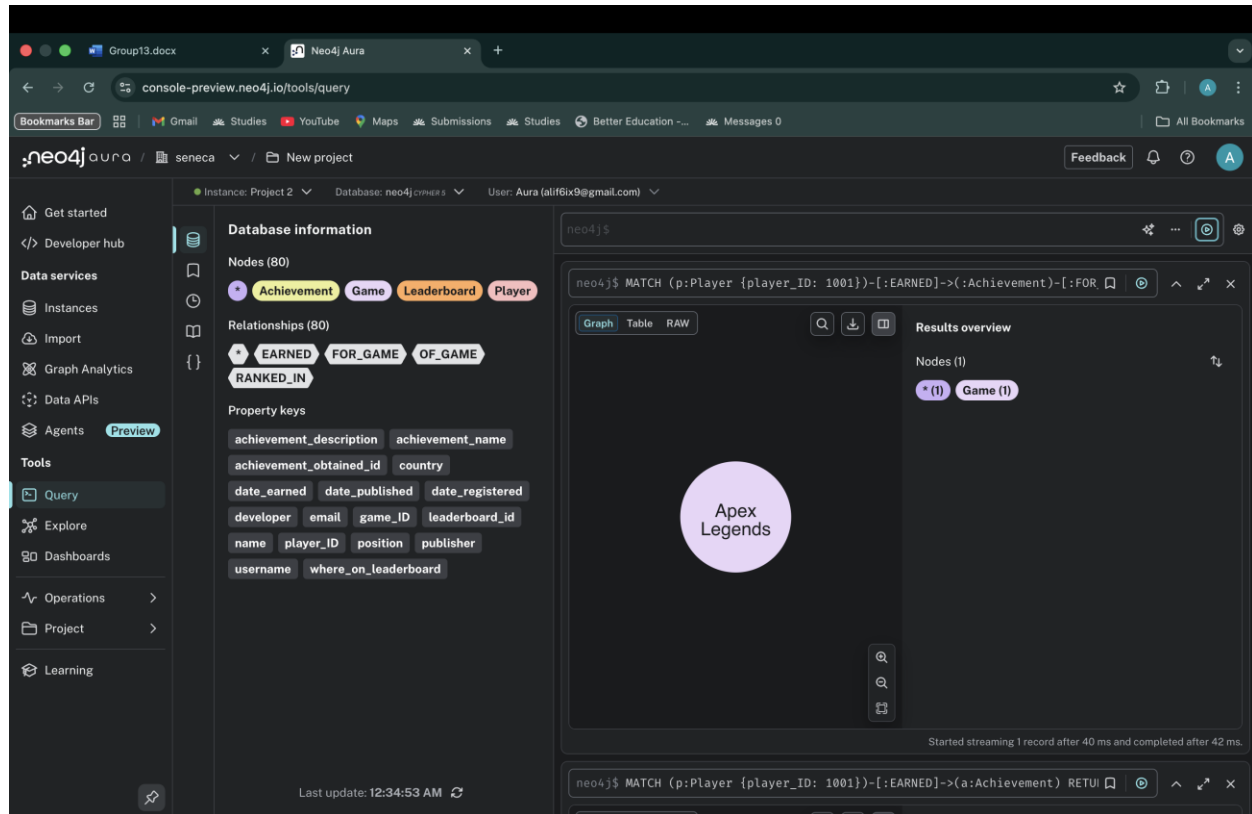
```
MATCH (p:Player {player_ID: 1001})-[:EARNED]->(a: Achievement)-[:FOR_GAME]->(g:Game) RETURN g;
```

Description:

This query performs a multi-hop traversal from a Player to their Achievements, and then to the Games where those achievements were earned.

It demonstrates pattern matching across multiple connected node types to retrieve related records.

Output:



Advanced query 2:

```
MATCH (p:Player)-[r:RANKED_IN]->(lb:Leaderboard)-[:OF_GAME]->(g:Game {game_ID: 5001})
RETURN p.username AS player, r.position AS rank ORDER BY rank ASC LIMIT 3;
```

Description:

This query retrieves the top three players ranked on the leaderboard for a specific game (game_ID: 5001).

It demonstrates how to combine filtering, relationship traversal, sorting, and limiting to return ranked leaderboard data.

Output:

The screenshot shows the Neo4j Aura console interface. On the left is a sidebar with navigation options like 'Get started', 'Developer hub', 'Data services', 'Instances', 'Import', 'Graph Analytics', 'Data APIs', 'Agents', 'Tools', 'Query', 'Explore', 'Dashboards', 'Operations', 'Project', and 'Learning'. The main area is divided into two panels. The left panel, titled 'Database information', shows 'Nodes (80)' with labels 'Achievement', 'Game', 'Leaderboard', and 'Player'. It also shows 'Relationships (80)' with labels 'EARNED', 'FOR_GAME', and 'OF_GAME'. Below this, 'Property keys' are listed for various nodes. The right panel shows a Cypher query: `neo4j$ MATCH (p:Player)-[:RANKED_IN]->(lb:Leaderboard)-[:OF_GAME]->(g:Game)`. Below the query, a table view shows results for 'player' and 'rank'. The table has three rows: 'PlayerOne' with rank 1, 'SharpShooter' with rank 2, and 'ExplorerX' with rank 3. A status message indicates 'Started streaming 3 records after 85 ms and completed after 93 ms.' Below the table, another query is shown: `neo4j$ MATCH (p:Player {player_ID: 1001})-[:EARNED]->(a:Achievement)-[:FOR_GAME]`. This query is followed by a 'Results overview' section showing 'Nodes (1)' with a count of 1 for 'Game (1)'. At the bottom of the console, there is a purple circle with the text 'Apex Legends'.

Advanced query 3:

```
MATCH (p:Player)-[:EARNED]->(a:Achievement)
```

```
RETURN p.username AS player, COUNT(a) AS total_achievements
```

```
ORDER BY total_achievements DESC;
```

Description:

This query uses aggregation to calculate how many achievements each player has earned. It demonstrates grouping results by player and applying the COUNT() function to summarize connected achievement nodes.

Output:

The screenshot displays the Neo4j Aura console interface. The left sidebar contains navigation options: Get started, Developer hub, Data services, Instances, Import, Graph Analytics, Data APIs, Agents (Preview), Tools, Query, Explore, Dashboards, Operations, Project, and Learning. The main panel is titled 'Database information' and shows the following details:

- Nodes (80):** Achievement, Game, Leaderboard, Player
- Relationships (80):** EARNED, FOR_GAME, OF_GAME, RANKED_IN
- Property keys:**
 - achievement_description, achievement_name
 - achievement_obtained_id, country
 - date_earned, date_published, date_registered
 - developer, email, game_ID, leaderboard_id
 - name, player_ID, position, publisher
 - username, where_on_leaderboard

The right panel shows two Cypher queries and their results.

Query 1:

```
neo4j$ MATCH (p:Player)-[:EARNED]->(a:Achievement) RETURN p.username AS p
```

Table View:

player	total_achievements
1 "PlayerOne"	1
2 "SharpShooter"	1
3 "ExplorerX"	1
4 "FlawlessWin"	1
5 "ItemHunter"	1
6 "Tactician"	1
7 "SurvivorPro"	1
8 "PuzzleKing"	1
9 "BossBreaker"	1
10 "SpeedRunner"	1

Query 2:

```
neo4j$ MATCH (p:Player)-[:RANKED_IN]->(lb:Leaderboard)-[:OF_GAME]->(g:Game)
```